

1.3 Program Structure

We'll have more to say later about program structure, but for now let's observe a few basics. A program consists of one or more functions; it may also contain global variables. (Our two example programs so far have contained one function apiece, and no global variables.) At the top of a source file are typically a few boilerplate lines such as `#include <stdio.h>`, followed by the definitions (i.e. code) for the functions. (It's also possible to split up the several functions making up a larger program into several source files, as we'll see in a later chapter.)

Each function is further composed of *declarations* and *statements*, in that order. When a sequence of statements should act as one (for example, when they should all serve together as the body of a loop) they can be enclosed in braces (just as for the outer body of the entire function). The simplest kind of statement is an *expression statement*, which is an expression (presumably performing some useful operation) followed by a semicolon. Expressions are further composed of *operators*, *objects* (variables), and *constants*.

C source code consists of several *lexical elements*. Some are words, such as `for`, `return`, `main`, and `i`, which are either *keywords* of the language (`for`, `return`) or *identifiers* (names) we've chosen for our own functions and variables (`main`, `i`). There are *constants* such as `1` and `10` which introduce new values into the program. There are *operators* such as `=`, `+`, and `>`, which manipulate variables and values. There are other punctuation characters (often called *delimiters*), such as parentheses and squiggly braces `{}`, which indicate how the other elements of the program are grouped. Finally, all of the preceding elements can be separated by *whitespace*: spaces, tabs, and the ``carriage returns" between lines.

The source code for a C program is, for the most part, ``free form." This means that the compiler does not care how the code is arranged: how it is broken into lines, how the lines are indented, or whether whitespace is used between things like variable names and other punctuation. (Lines like `#include <stdio.h>` are an exception; they must appear alone on their own lines, generally unbroken. Only lines beginning with `#` are affected by this rule; we'll see other examples later.) You can use whitespace, indentation, and appropriate line breaks to make your programs more readable for yourself and other people (even though the compiler doesn't care). You can place explanatory *comments* anywhere in your program--any text between the characters `/*` and `*/` is ignored by the compiler. (In fact, the compiler pretends that all it saw was whitespace.) Though comments are ignored by the compiler, well-chosen comments can make a program *much* easier to read (for its author, as well as for others).

The usage of whitespace is our first *style* issue. It's typical to leave a blank line between different parts of the program, to leave a space on either side of operators such as `+` and `=`, and to *indent* the bodies of loops and other control flow constructs. Typically, we arrange the indentation so that the subsidiary statements controlled by a loop statement (the ``loop body," such as the `printf` call in our second example program) are all aligned with each other and placed one tab stop (or some consistent number of spaces) to the right of the controlling statement. This indentation (like all whitespace) is not required by the compiler, but it makes programs *much* easier to read. (However, it can also be misleading, if used incorrectly or in the face of inadvertent mistakes. The compiler will decide what ``the body of the loop" is based on its own rules, not the indentation, so if the indentation does not match the compiler's interpretation, confusion is inevitable.)

To drive home the point that the compiler doesn't care about indentation, line breaks, or other whitespace, here are a few (extreme) examples: The fragments

```
for(i = 0; i < 10; i = i + 1)
    printf("%d\n", i);
```

and

```
for(i = 0; i < 10; i = i + 1) printf("%d\n", i);
```

and

```
for(i=0;i<10;i=i+1)printf("%d\n",i);
```

and

```
    for(i = 0; i < 10; i = i + 1)
printf("%d\n", i);
```

and

```
for      (      i
=        0      ;
i        <      10
;        i      =
i        +      1
)        printf (
"%d\n"    ,      i
)        ;
```

and

```
    for
    (i=0;
    i<10;i=
    i+1)printf
    ("%d\n", i);
```

are all treated exactly the same way by the compiler.

Some programmers argue forever over the best set of "rules" for indentation and other aspects of programming style, calling to mind the old philosopher's debates about the number of angels that could dance on the head of a pin. Style issues (such as how a program is laid out) *are* important, but they're not something to be too dogmatic about, and there are also other, deeper style issues besides mere layout and typography. Kernighan and Ritchie take a fairly moderate stance:

Although C compilers do not care about how a program looks, proper indentation and spacing are critical in making programs easy for people to read. We recommend writing only one statement per line, and using blanks around operators to clarify grouping. The position of braces is less important, although people hold passionate beliefs. We have chosen one of several popular styles. Pick a style that suits you, then use it consistently.

There is some value in having a reasonably standard style (or a few standard styles) for code layout. Please don't take the above advice to "pick a style that suits you" as an invitation to invent your own brand-new style. If (perhaps after you've been programming in C for a while) you have specific objections to specific facets of existing styles, you're welcome to modify them, but if you don't have any particular leanings, you're probably best off copying an existing style at first. (If you want to place your own stamp of originality on the programs that you write, there are better avenues for your creativity than inventing a bizarre layout; you might instead try to make the logic easier to follow, or the user interface easier to use, or the code freer of bugs.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail](#) [feedback](#)